

Prokoders Flutter Task - Documentation

1. Overview

This document outlines the implementation of the Prokoders Login Task. The app is built using Flutter and integrates with Firebase for authentication, implementing clean code, SOLID and DRY principles, and Design Pattern (Singleton and factory and repository). It includes essential features such as login, signup, and item management.

2. Features

- User Authentication (Firebase Email/Password)
- Login & Signup Screens
- Firebase integration
- Item List Display (with pagination)
- Add Item via BottomSheet Form
- State Management using Provider
- Responsive UI with ScreenUtil
- Error Handling with SnackBars

3. Firebase Authentication

Authentication is handled via FirebaseAuth.

AuthProvider manages login, signup, password visibility toggle, and loading states.

Input validation is implemented for email and password formats.

4. Home Screen and Item Management

HomeScreen displays a paginated list of items using ListView.

Items are fetched from a backend API via the ItemRepository.

Provider listens to scroll events for pagination.

Items can be added using a BottomSheet form.

5. Error Handling

All async operations (login, signup, add item) are wrapped in try-catch blocks.

Errors are displayed using customSnackBar.

Additional handling for unmounted widget errors during context usage is implemented.

6. Optimization & Build

- Forced LTR layout direction regardless of device language.

7. UI/UX Enhancements

- Implemented Lottie animations for loading states.
- Used FancyShimmerImage for item thumbnails.
- Improved BottomSheet behavior to avoid being hidden by the keyboard.
- Implemented custom styling (Allerta Stencil font, modern spacing).

8. Tools & Packages Used

- firebase_auth
- provider
- Lottie
- flutter_screenutil
- fancy_shimmer_image
- google_fonts
- dio

9. Data structure design pattern

Singleton Pattern

Implement a Singleton pattern for managing a user session.

Explanation: Create one instance of UserSession class in the whole application life.

Usage: maintaining login state throughout the app, by set user when sign in, and clear it when signout.

How the singleton Implemented:

By create a class UserSession with getter and setter, with create on instance through internal constructor, and set user when sign in, and get it from any screen and page, and clear it* when sign out

*:Clear the information of the user not the object itself

Factory Pattern

Implement a Factory pattern that provides a centralized way to create instances of classes without exposing the instantiation logic to the client. It promotes loose coupling by separating the object creation from its usage..

Usage: Create a factory for generating different types of data objects (Product).

How the Factory Implemented

Create a class called ProductFactory that two static methods:

- 1- createWithParams(...)
Creates a custom Product by passing values like id, title, and price.

- 2- createFromJson(Map<String, dynamic>)
Converts a JSON map into a Product object.

10. SOLID and DRY principles

- Refactored AUTH Provider and Items Provider to follow SRP by moving validation and error handling logic into Object Validator and Ui States classes.
- Applied DIP by injecting Items Repository interface instead of get items directly
- Removed duplicate validation code using shared methods in Input Validator (DRY).

11. Common Errors Solved

- Handling context after dispose
- RTL problem in Devices that have Arabic language settings

11. Improvements Possible

- Add unit testing
- Localization
- Dark mode & Light mode

12. Time Frames

- Part 1: 8 hours
- Part 2: 8 hours
- Part 3&4: two hours

Done at 2025-05-01

Github Repo: https://github.com/aboodalh1/ProKoders_Task

AbdAllah Alharisi